

Sistema Avanzado de Observabilidad y Remediación Automática en Kubernetes usando eBPF

Documentación técnica del Trabajo de Fin de Grado

Autor: Ricardo Ruiz

Fecha del documento: 11 de mayo de 2026

Versión: 1.0

Resumen

Este Trabajo de Fin de Grado presenta un sistema completo de observabilidad y remediación automática para clusters de Kubernetes que opera a nivel de **kernel** mediante programas **eBPF**. A diferencia de las herramientas tradicionales (cAdvisor, kube-state-metrics, Prometheus node-exporter), el sistema captura métricas que solo son accesibles desde dentro del kernel: la *latencia de I/O de bloque* y la *latencia de la cola de ejecución (Run Queue Latency)* de los procesos. Estas dos métricas son los indicadores más precisos de saturación de disco y CPU respectivamente, y permiten detectar cuellos de botella que el sistema de scheduling y autoscaling estándar de Kubernetes no llega a ver.

El sistema se compone de tres bloques: (1) programas eBPF escritos en C con CO-RE que instrumentan el kernel; (2) un agente recolector en Go que mapea los *cgroup IDs* a Pods reales y expone las métricas mediante un endpoint Prometheus; y (3) un **Operador de Kubernetes** escrito con Kubebuilder que define un Custom Resource `IORemediationPolicy` y, según las métricas, ejecuta acciones de remediación: escalado vertical *in-place*, migración de almacenamiento o expulsión con *taint* del nodo afectado.

Índice

1. [Introducción y motivación](#)
2. [Objetivos](#)
3. [Estado del arte y contexto técnico](#)
4. [Arquitectura general del sistema](#)
5. [Componente 1 — Instrumentación eBPF](#)
6. [Componente 2 — Agente Recolector \(Go\)](#)
7. [Componente 3 — Operador de Kubernetes](#)
8. [Flujo de ejecución end-to-end](#)
9. [Despliegue y operación](#)
10. [Stack tecnológico](#)

11.[Decisiones de diseño](#)

12.[Limitaciones conocidas y trabajo futuro](#)

13.[Apéndices](#)

1. Introducción y motivación

1.1 Contexto

Kubernetes se ha consolidado como el orquestador de contenedores de referencia. Sin embargo, su modelo de planificación y autoscaling se apoya casi exclusivamente en métricas *resource-based* de alto nivel: uso de CPU expresado como cuota cgroup, consumo de memoria RSS y, opcionalmente, métricas custom expuestas mediante el adaptador de métricas. Estas señales tienen una limitación fundamental: informan de **consumo**, no de **saturación**.

Un pod puede tener un uso medio de CPU del 30 % y, simultáneamente, encontrarse degradado porque sus hilos están esperando largos periodos en la cola de ejecución debido a contención con vecinos ruidosos (*noisy neighbors*). Algo análogo ocurre con el disco: el porcentaje de utilización medido por herramientas tradicionales no detecta cuándo las peticiones de I/O se están encolando y servir cada una tarda decenas de milisegundos.

1.2 Por qué eBPF

eBPF (*extended Berkeley Packet Filter*) permite ejecutar código verificado y sandboxed dentro del kernel de Linux, asociado a puntos de instrumentación como *tracepoints*, *kprobes*, *uprobes* y eventos de scheduling. Los programas eBPF se ejecutan en el contexto del propio evento, con muy bajo overhead, y comparten datos con el espacio de usuario mediante *maps*. Esto permite medir con resolución

de nanosegundos cosas como el tiempo entre que una petición de bloque es emitida y completada por el dispositivo, o el tiempo que un hilo pasa esperando en la cola de ejecución antes de obtener una CPU, todo ello sin recompilar el kernel ni cargar módulos.

1.3 El problema concreto

Las decisiones de remediación que toma Kubernetes (*HPA*, *VPA*, *descheduler*) dependen de las métricas que recibe. Si las métricas no reflejan saturación real, las decisiones llegan tarde o son erróneas. Por ejemplo:

- El **HPA** escala horizontalmente cuando el uso de CPU pasa de un umbral, pero ese umbral puede no dispararse aunque los hilos del pod estén penalizados por contención.
- El **VPA** propone nuevos *requests*, pero típicamente requiere reinicio del pod, lo que en cargas con estado es disruptivo.
- El **scheduler** elige nodos en base a *requests* declarados, no a la saturación real del nodo destino.

El sistema propuesto cierra ese bucle: detecta saturación de I/O o de CPU al nivel del kernel, lo publica como métricas Prometheus, y actúa automáticamente según políticas declarativas.

2. Objetivos

Objetivo principal

Desarrollar un sistema capaz de detectar cuellos de botella y saturación de recursos a nivel de kernel y aplicar acciones de remediación de forma automática en un cluster de Kubernetes.

Objetivos específicos

1. Implementar dos programas eBPF (uno para I/O de bloque, otro para Run Queue Latency) que agreguen estadísticas por `cgroup_id` sin overhead

significativo.

2. Construir un agente en Go que lea los mapas eBPF, resuelva `cgroup_id` a Pods reales y exponga las métricas en formato Prometheus.
3. Diseñar un CRD (`IORemediationPolicy`) que permita expresar políticas de remediación de forma declarativa y un controlador Kubernetes que las evalúe periódicamente.
4. Implementar tres acciones de remediación: *In-Place Vertical Scaling* de CPU, migración de *StorageClass* y *EvictAndTaint* del nodo saturado.
5. Orquestar el despliegue local mediante un script que cargue los programas eBPF, ancle sus mapas y lance los componentes de espacio de usuario.

3. Estado del arte y contexto técnico

3.1 eBPF y CO-RE

eBPF nació como una extensión del BPF clásico para filtrado de paquetes. Hoy en día es la base de proyectos como Cilium, Falco, BCC, bpftrace o Pixie. Un programa eBPF consta de:

- Un **tipo de programa** que define a qué evento se engancha (kprobe, tracepoint, tp_btf, XDP, sched_cls, etc.).
- Uno o varios **mapas** (hash, array, ringbuffer, LRU...) que actúan como canal de comunicación entre kernel y userspace.
- El propio **código BPF** compilado a bytecode, verificado por el verificador del kernel antes de cargarse.

CO-RE (Compile Once - Run Everywhere) resuelve un problema antiguo: los offsets de las estructuras del kernel cambian entre versiones. Con CO-RE el compilador (Clang con BTF) emite relocations que se resuelven en el destino usando la información BTF del kernel en ejecución. Esto permite distribuir un único binario eBPF y que funcione en cualquier kernel con BTF habilitado.

3.2 cgroups v2

Kubernetes (con runtimes como containerd o CRI-O) coloca cada contenedor en un *cgroup* específico. En **cgroups v2** cada cgroup tiene un identificador entero único de 64 bits (el inodo del directorio en cgroupfs). Desde un programa eBPF se puede obtener el cgroup del proceso actual con `bpf_get_current_cgroup_id()` o navegando por `task_struct`. La correlación `cgroup_id` → contenedor → Pod se hace desde espacio de usuario leyendo la jerarquía de `/sys/fs/cgroup`.

3.3 Métricas de saturación: definición operativa

Métrica	Definición	Tracepoints utilizados
Block I/O latency	Tiempo entre emisión y compleción de cada <code>struct request</code> .	<code>block_rq_issue</code> , <code>block_rq_complete</code>
Run Queue latency	Tiempo que un hilo pasa en estado <i>runnable</i> esperando ser elegido por el scheduler.	<code>sched_wakeup</code> , <code>sched_wakeup_new</code> , <code>sched_switch</code>

3.4 Kubernetes: CRDs y Operator pattern

Un **Custom Resource Definition** extiende la API de Kubernetes con nuevos tipos. Un **Operador** es un controlador que reconcilia el estado real del cluster con el deseado expresado en esos recursos. El proyecto utiliza **Kubebuilder** + **controller-runtime**, que generan el scaffolding del CRD, las webhooks, los esquemas OpenAPI y un loop de reconciliación estándar.

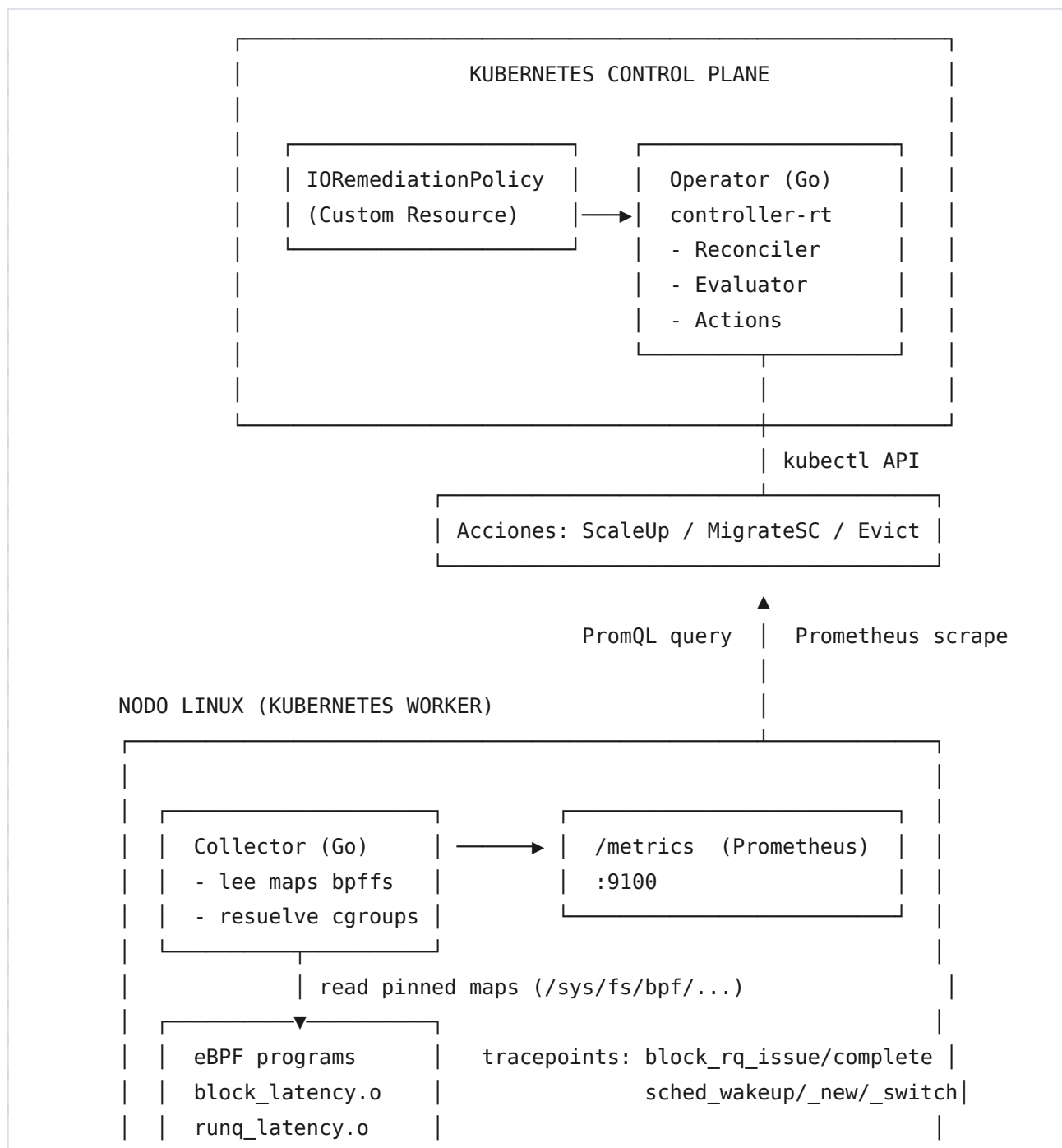
3.5 In-Place Pod Vertical Scaling

A partir de Kubernetes 1.27 (*feature gate* `InPlacePodVerticalScaling`, en beta) es posible cambiar los *resources* de un pod sin reiniciarlo. La actualización debe ir contra el subrecurso `resize` del Pod, no contra el recurso principal (los campos `spec.containers[*].resources` permanecen inmutables vía un update normal).

4. Arquitectura general del sistema

4.1 Visión de alto nivel

El sistema se descompone en tres planos: **kernel**, **espacio de usuario del nodo** y **plano de control de Kubernetes**. Todo el flujo es unidireccional desde el evento crudo del kernel hasta la decisión de remediación.



4.2 Estructura del repositorio

Directorio	Contenido
<code>ebpf/latency/</code>	Fuentes C de los programas eBPF.
<code>ebpf/*.o</code>	Objetos BPF ya compilados, listos para cargar con <code>bpftool</code> .
<code>collector/</code>	Agente recolector en Go (lee maps, expone Prometheus, dashboard TUI opcional).
<code>operator/</code>	Operador de Kubernetes (Kubebuilder): CRD, controller, evaluator, acciones.
<code>start_system</code>	Script Bash de despliegue local.
<code>CLAUDE.md</code>	Contexto técnico del proyecto.

5. Componente 1 — Instrumentación eBPF

5.1 Visión general

Dos programas independientes, compilados de C a bytecode BPF con Clang + BTF, anclados como objetos persistentes en el filesystem BPF (`/sys/fs/bpf/`):

- `block_latency.o` — instrumenta el subsistema de bloque del kernel.
- `runq_latency.o` — instrumenta el scheduler.

Ambos exponen un mapa `hash` agregado por `cgroup_id` que el collector lee periódicamente desde espacio de usuario.

5.2 `block_latency.c` — Latencia de I/O de bloque

Objetivo

Medir el tiempo real (en nanosegundos) que tarda cada petición de disco desde que el kernel la envía al driver del dispositivo hasta que ese mismo dispositivo notifica la compleción. Agrega resultados por cgroup para imputar la latencia al contenedor responsable.

Mapas

Mapa	Tipo	Clave	Valor	Propósito
<code>start_map</code>	HASH (32 768)	u64 (puntero a <code>struct request</code>)	<code>start_val_t</code> (ts_ns, bytes, cgroup_id)	Correlaciona <i>issue</i> con <i>complete</i> .
<code>stats_by_cgrou p</code>	HASH (16 384)	u64 (cgroup_id)	<code>cgroup_io_stats_t</code>	Acumulado de IOPS, latencia, bytes y errores por cgroup.

Programas

Se enganchan a los tracepoints BTF `tp_btf/block_rq_issue` y `tp_btf/block_rq_complete`. Al usar `tp_btf + BPF_PROG` se obtiene acceso tipado a los argumentos (`struct request *rq`), eliminando la necesidad de redefinir manualmente el contexto del tracepoint y haciendo el código portable entre versiones de kernel gracias a CO-RE.

Issue

```
SEC("tp_btf/block_rq_issue")
int BPF_PROG(block_rq_issue, struct request *rq) {
    struct start_val_t val = {};
    u64 key = (u64)rq;
    val.ts_ns    = bpf_ktime_get_ns();
    val.bytes    = BPF_CORE_READ(rq, __data_len);
    val.cgroup_id = bpf_get_current_cgroup_id();
    bpf_map_update_elem(&start_map, &key, &val, BPF_ANY);
    return 0;
}
```

```
}
```

La clave es el **puntero a struct request**: dentro del kernel cada request es única e identificable por su dirección, lo que elimina las colisiones que sufre la aproximación clásica (`dev`, `sector`, `nr_sector`).

Complete

```
SEC("tp_btf/block_rq_complete")
int BPF_PROG(block_rq_complete, struct request *rq, int error, unsigned int
nr_bytes) {
    u64 key = (u64)rq;
    struct start_val_t *start = bpf_map_lookup_elem(&start_map, &key);
    if (!start) return 0;
    u64 delta = bpf_ktime_get_ns() - start->ts_ns;
    /* upsert en stats_by_cgroup y __sync_fetch_and_add para evitar race */
    bpf_map_delete_elem(&start_map, &key);
    return 0;
}
```

Las actualizaciones de los contadores se hacen con `__sync_fetch_and_add` (atomicidad por hardware) porque varias CPUs pueden completar requests del mismo cgroup en paralelo.

5.3 `runq_latency.c` — Latencia de la cola de ejecución

Objetivo

Medir el tiempo que cada hilo pasa en estado *runnable* antes de ser elegido para ejecutarse. Es la métrica más directa de saturación de CPU.

Mapas

Mapa	Tipo	Clave	Valor	Propósito
<code>start_map</code>	HASH (16 384)	<code>runq_key_t =</code> (<code>pid</code> , <code>start_time</code>)	<code>u64 (ts_ns)</code>	Timestamp del último wakeup/preem

pt.

<code>runq_stats_by_cgroup</code>	<code>HASH</code>	<code>u64</code>	<code>cgroup_runq_stats_</code>	Acumulado de
	<code>(8</code>	<code>(cgroup_id)</code>	<code>t</code>	eventos y
	<code>192)</code>			latencia por
				cgroup.

Lógica

Se enganchan tres programas:

- `tp_btf/sched_wakeup`: el task vuelve a runnable porque ha sido despertado.
- `tp_btf/sched_wakeup_new`: idéntico, para procesos recién creados.
- `tp_btf/sched_switch`: se elige otro task para correr; aquí se cierra el delta.

Adicionalmente, dentro de `sched_switch`, si el task que sale (`prev`) sigue en estado `TASK_RUNNING` (es decir, ha sido *preempted*, no se ha dormido), se vuelve a abrir su ventana de medición. Esto captura la latencia de re-encolado, no solo la de wakeup, y es lo que diferencia esta implementación de versiones más ingenuas.

Clave compuesta

La clave del `start_map` incluye `start_time` del `task_struct` además del PID, evitando colisiones cuando el kernel reutiliza PIDs en sistemas con churn alto de procesos cortos.

CO-RE

Todos los accesos a campos de `task_struct` usan `BPF_CORE_READ`, así el binario funciona en cualquier kernel con BTF independientemente de los offsets exactos.

6. Componente 2 — Agente Recolector (Go)

6.1 Responsabilidades

1. Cargar los mapas BPF pinned y mantener un loop de lectura periódica.

2. Resolver `cgroup_id` a la ruta del cgroup en `/sys/fs/cgroup` y clasificarla.
3. Extraer metadatos de Kubernetes (Pod UID, runtime, container ID) desde la ruta del cgroup.
4. Exponer las métricas en formato Prometheus en `:9100/metrics`.
5. Opcionalmente, mostrar un *dashboard* en consola para depuración.

6.2 Estructura de ficheros

Fichero	Función
<code>main.go</code>	Bootstrap, HTTP server, loop principal, gestión de señales.
<code>cgroup_resolver.go</code>	Walk de <code>/sys/fs/cgroup</code> + <code>name_to_handle_at</code> para obtener cgroup IDs.
<code>cgroup_classifier.go</code>	Clasifica rutas como Host/Systemd/Container/Kubernetes/Unknown.
<code>kube_path_parser.go</code>	Regex para extraer Pod UID, container ID y runtime de las rutas.
<code>prom_metrics.go</code>	Exposición Prometheus con CounterVec y agregación por <code>pod_uid</code> .

6.3 Resolución de `cgroup_id`

La forma estándar de obtener el `cgroup_id` de una ruta es la syscall `name_to_handle_at` con flag 0 sobre el sistema de archivos cgroup. El handle resultante contiene exactamente 8 bytes que codifican el id en el orden de bytes nativo del CPU. El resolver hace un `WalkDir` de toda la jerarquía `/sys/fs/cgroup` al arrancar y mantiene un mapa `id → path`.

Para evitar reconstruir todo el árbol cada 5 s, el rebuild se realiza solo cuando aparece un `cgroup_id` que no está en la caché (lazy refresh).

6.4 Clasificación y parsing

La función `ClassifyCgroupPath` devuelve un enumerado `CgroupKind` según la ruta:

- `kubepods*` → *kubernetes*

- `containerd / docker / cri-containerd` → *container*
- `*.slice, init.scope` → *systemd*
- `/sys/fs/cgroup` raíz → *host*

`ParseKubePath` aplica dos regex (formato con guiones y formato con underscores) para extraer el Pod UID, y una tercera (64 hex) para el container ID.

6.5 Exposición Prometheus

Las métricas son `CounterVec`, no `GaugeVec`: los mapas BPF acumulan totales monótonos, y `rate()/increase()` en PromQL solo funcionan sobre counters. Para evitar el race entre `Reset()` y `Set()`, el collector mantiene en memoria el snapshot anterior por cada label set y solo aplica **deltas positivos** con `Add()`. Cuando un cgroup deja de aparecer en un ciclo, se borran sus series para que la cardinalidad no crezca indefinidamente.

Métricas expuestas

Nombre	Tipo	Descripción
<code>ebpf_block_io_count_total</code>	Counter	Operaciones de bloque por pod.
<code>ebpf_block_io_bytes_total</code>	Counter	Bytes transferidos por pod.
<code>ebpf_block_io_latency_ns_total</code>	Counter	Latencia acumulada en ns.
<code>ebpf_block_io_errors_total</code>	Counter	Errores de I/O por pod.
<code>ebpf_block_io_avg_latency_ns</code>	Gauge	Latencia media instantánea por pod.
<code>ebpf_cpu_runq_events_total</code>	Counter	Eventos de scheduling por pod.
<code>ebpf_cpu_runq_latency_ns_total</code>	Counter	Latencia acumulada de runqueue.

Labels: kind, pod_uid.

6.6 Dashboard TUI

Si la variable `COLLECTOR_DEBUG_TUI=1` está presente, el collector imprime cada 5 s una tabla ANSI con los stats por cgroup. Útil en desarrollo, desactivado por defecto para que el binario sea amigable con `systemd` y similares.

7. Componente 3 — Operador de Kubernetes

7.1 Custom Resource: `IORemediationPolicy`

Definido en `operator/api/v1alpha1/ioremediationpolicy_types.go`, scope *Namespaced*, grupo `autoremediation.tfg.local`. Schema OpenAPI generado con kubebuilder.

Spec

Campo	Tipo	Descripción
<code>targetPodSelector</code>	<code>LabelSelector</code>	Pods a los que aplica la política.
<code>prometheusEndpoint</code>	<code>string</code>	URL del Prometheus (p. ej. <code>http://prometheus:9090</code>).
<code>metricType</code>	<code>enum {IO, CPU}</code>	Qué saturación se está vigilando. Default IO.
<code>latencyThreshold</code>	<code>string (duration)</code>	Umbral de latencia promedio (p. ej. <code>50ms</code>).
<code>evaluationWindow</code>	<code>string (PromQL)</code>	Ventana para el

	duration)	rate() (p. ej. 5m).
action	enum {EvictAndTaint, MigrateStorageClass , ScaleUp}	Acción a ejecutar.
evictAndTaintConfig.taintDuration	Duration	Vida útil del taint aplicado.
migrateStorageConfig.targetStorageClass	string	StorageClass destino para la migración.
scaleUpConfig.cpuStepPercent	int32	% de incremento por iteración.
scaleUpConfig.maxCpuLimit	string (Quantity)	Límite absoluto, e. g. 2000m.

Status

Lista estándar de `metav1.Condition`. El reconciler escribe condiciones *Available*, *Progressing* o *Degraded* tras cada ciclo, lo que permite a `kubectl describe` mostrar lo que está haciendo el operador.

7.2 Reconciler

Cada vez que cambia un CR (o periódicamente cada `EvaluationWindow`) se ejecutan los siguientes pasos:

1. Carga el CR (`r.Get`).
2. Construye un evaluador y consulta Prometheus para detectar pods saturados.
3. Construye el objeto de acción correspondiente (`ScaleUp`, `MigrateStorageClass` o `EvictAndTaint`) con sus parámetros y, en el caso de `ScaleUp`, un fallback a `EvictAndTaint`.
4. Para cada pod saturado, ejecuta la acción y cuenta los fallos.
5. Actualiza `Status.Conditions`.
6. Devuelve `RequeueAfter: EvaluationWindow` para volver a evaluar.

7.3 Evaluator: PrometheusEvaluator

Encapsula la consulta PromQL y la traducción `pod_uid` → `Pod.Name`. Pasos:

1. Validar `evaluationWindow` y `latencyThreshold`.
2. Construir la query según `MetricType`:

```
(rate(latency_ns_total[w]) / ignoring() (rate(events_total[w]) > 0)) > THRESHOLD_NS
```

3. Ejecutar la query en el endpoint configurado.
4. Quedarse con los `pod_uid` que aparecen en el vector resultado.
5. Listar pods que casan con `targetPodSelector` en el namespace del CR y intersectar por UID.

La unión por UID es segura porque los UIDs de Kubernetes son UUIDs globalmente únicos. El filtrado por namespace se hace en este paso, no en la query PromQL.

7.4 Acciones de remediación

7.4.1 ScaleUp (In-Place Vertical Scaling)

Sube el *limit* de CPU de cada contenedor en un porcentaje, con tope absoluto, sin reiniciar el pod. Detalles:

- Baseline: el `Limit` actual; si no existe, el `Request`. Si ninguno está definido, el contenedor se omite (evita el bug clásico de "downgrade a 100m").
- El step se calcula como `currentMillis * CPUStepPercent / 100` con un mínimo de 10 m.
- Se aplica como **patch sobre el subrecurso `resize`**, requisito de la API de `InPlacePodVerticalScaling`.
- Si todos los contenedores ya están al máximo, ejecuta la *fallback action* (`EvictAndTaint`).

7.4.2 MigrateStorageClass

Migra los PVCs del Deployment al que pertenece el Pod hacia una *StorageClass* más

rápida. Detalles y salvaguardas:

- Verifica que la `StorageClass` destino existe antes de tocar nada.
- Recorre la cadena de propietarios `Pod` → `ReplicaSet` → `Deployment`. Si el `Pod` no está gestionado por un `Deployment` se rechaza explícitamente la acción.
- Procesa **todos** los volúmenes con `PVC` del `Deployment`.
- Para cada `PVC`: si ya está en la `StorageClass` destino, se salta; si tiene datos (`Status.Phase != ClaimPending`) se rechaza (no copia datos sin `VolumeSnapshot`).
- Crea el nuevo `PVC` con `DataSource` apuntando al original y patchea el `Deployment` para apuntar al nuevo nombre.

7.4.3 EvictAndTaint

Quita el pod del nodo saturado y marca el nodo con un taint `NoSchedule` para impedir que el scheduler vuelva a colocar pods de la misma clase. Detalles:

- Taint key parametrizable; default `autoremediation.tfg.local/saturation`.
- Si la política define `taintDuration`, el taint se purga en reconciliaciones posteriores cuando ha vencido (`TimeAdded + duration ≤ now`).
- La expulsión usa el subrecurso `pods/eviction` (respeta `PodDisruptionBudgets`).

7.5 Permisos (RBAC)

El controlador necesita acceso a múltiples recursos. Los markers de kubebuilder generan un `Role/ClusterRole` con:

```
+kubebuilder:rbac:groups="",resources=pods,verbs=get;list;watch;update;patch
+kubebuilder:rbac:groups="",resources=pods/eviction,verbs=create
+kubebuilder:rbac:groups="",resources=nodes,verbs=get;list;watch;update;patch
+kubebuilder:rbac:groups="",resources=persistentvolumeclaims,verbs=get;list;watch;
create;update;patch
+kubebuilder:rbac:groups=storage.k8s.io,resources=storageclasses,verbs=get;list;wa
tch
+kubebuilder:rbac:groups=apps,resources=deployments;replicasets,verbs=get;list;wat
ch;update;patch
```

8. Flujo de ejecución end-to-end

1. **Evento en el kernel:** un proceso de un pod inicia una I/O o entra en runqueue.
2. **Programa eBPF:** el tracepoint correspondiente dispara, se anota el timestamp y se identifica el cgroup_id.
3. **Evento final** (complete / sched_switch): se calcula el delta y se actualiza el contador agregado por cgroup en un `BPF_MAP_TYPE_HASH`.
4. **Tick del collector:** cada 5 s recorre los mapas, resuelve cgroup_id, agrega por pod_uid y publica el delta como incremento del Counter Prometheus.
5. **Scrape de Prometheus:** la instancia Prometheus consulta `:9100/metrics` según su `scrape_interval`.
6. **Reconcile del operador:** cada `EvaluationWindow`, el reconciler consulta `rate()` sobre la métrica relevante; si supera el umbral se obtienen los pod_uid saturados.
7. **Acción:** se ejecuta la acción configurada en el CR sobre cada pod.
8. **Status update:** el controlador deja constancia del resultado en `policy.status.conditions`.

9. Despliegue y operación

9.1 Prerequisitos

- Kernel Linux $\geq$ 5.10 con BTF habilitado (`CONFIG_DEBUG_INFO_BTF=y`).
- `bpftool`, `clang`, `llvm`, headers del kernel.
- Go $\geq$ 1.21, Kubernetes $\geq$ 1.27 (para In-Place Vertical Scaling).
- Prometheus accesible desde el plano de control.

9.2 Compilación

```
# eBPF
cd ebpf/latency
clang -O2 -target bpf -g \
    -I/usr/include/$(uname -m)-linux-gnu \
    -c block_latency.c -o ../block_latency.o
clang -O2 -target bpf -g \
    -I/usr/include/$(uname -m)-linux-gnu \
    -c runq_latency.c -o ../runq_latency.o

# Collector
cd collector && go build -o collector .

# Operator
cd operator && make build
```

9.3 Script `start_system`

El script orquesta el arranque local del nodo:

1. Habilita `br_netfilter` y los `sysctl` que pide Kubernetes.
2. Desactiva swap (requisito de kubelet).
3. Reinicia `containerd` y kubelet.
4. Carga ambos objetos BPF con `bpftool prog loadall ... autoattach pinmaps`.
5. Comprueba que se ha pineado al menos un objeto por programa.
6. (Opcional con `./start_system yes`) lanza Prometheus.
7. Lanza el collector con `nohup` en background y comprueba el endpoint HTTP.

9.4 Despliegue del operador

```
cd operator
make install          # CRD en el cluster
make deploy IMG=<tu-registry>/operator:v0.1
kubectl apply -f config/samples/
```

9.5 Ejemplo de `IORemediationPolicy`

```
apiVersion: autoremediation.tfg.local/v1alpha1
kind: IORemediationPolicy
metadata:
  name: io-protect-fastpath
  namespace: default
spec:
  targetPodSelector:
    matchLabels:
      tier: storage
  prometheusEndpoint: http://prometheus.monitoring:9090
  metricType: IO
  latencyThreshold: "50ms"
  evaluationWindow: "5m"
  action: MigrateStorageClass
  migrateStorageConfig:
    targetStorageClass: nvme-ssd
```

10. Stack tecnológico

Capa	Tecnología	Versión recomendada
Kernel / instrumentación	Linux + eBPF + BTF + CO-RE	≥ 5.10
Lenguaje eBPF	C (Clang BPF backend)	Clang ≥ 14
Carga eBPF	<code>bpftool</code>	libbpf 1.x
Espacio de usuario	Go	≥ 1.21
eBPF en Go	<code>github.com/cilium/ebpf</code>	v0.14+
Métricas	Prometheus <code>client_golang</code>	v1.18+
Backend de métricas	Prometheus	v2.50+
Orquestador	Kubernetes	≥ 1.27

SDK del operador	Kubebuilder + controller-runtime	kubebuilder $\geq$ 3.14
Scripts	Bash	$\geq$ 5

11. Decisiones de diseño

11.1 Counter vs Gauge en Prometheus

La elección de `CounterVec` sobre `GaugeVec` es deliberada: los mapas BPF acumulan totales monótonos y `rate()` solo es semánticamente correcto sobre counters. Esta elección también evita el race entre `Reset()` y `Set()` durante un scrape, que provocaría muestras vacías intermitentes en el dashboard.

11.2 Clave por puntero a `struct request`

La clave clásica (`dev`, `sector`, `nr_sector`) sufre colisiones en cargas con muchas operaciones simultáneas sobre el mismo dispositivo. El puntero a `struct request` es único, atómico y se obtiene gratis desde el contexto BTF del tracepoint.

11.3 Refresco lazy del resolver de cgroups

Reconstruir `/sys/fs/cgroup` con `filepath.WalkDir` cada 5 s es caro en nodos con muchos cgroups. El resolver se reconstruye solo cuando aparece un `cgroup_id` desconocido — el caso común es no hacer trabajo extra.

11.4 In-Place Vertical Scaling vs HPA

El sistema escala verticalmente sin reinicio porque muchas cargas saturadas son *stateful* (BBDD, colas, caches) donde un reinicio agrava el problema. HPA queda fuera del scope: añadir réplicas no soluciona el problema cuando la latencia es interna del pod.

11.5 Rechazo conservador en `MigrateStorageClass`

Crear un PVC nuevo sin copiar datos es destructivo. La acción se niega explícitamente a operar sobre PVCs con datos. La copia real con `VolumeSnapshot` queda como evolución futura.

11.6 Fallback en `ScaleUp`

Cuando un pod ya está al máximo configurado, escalar no aporta nada. En vez de fallar silenciosamente se ejecuta una acción configurable (por defecto `EvictAndTaint`): así el sistema sigue intentando aliviar la saturación.

12. Limitaciones conocidas y trabajo futuro

Limitaciones

- **Cardinalidad por `pod_uid`:** en clusters con churn alto la cardinalidad total puede ser elevada. Mitigada mediante borrado de series cuando los pods desaparecen, pero sigue siendo un riesgo a vigilar.
- **Granularidad de `cgroup` vs proceso:** la agregación es por `cgroup`. Si dos contenedores del mismo pod tienen perfiles muy distintos, la métrica del pod los mezcla.
- **Solo Deployments en `MigrateStorageClass`:** `StatefulSets` y `DaemonSets` no están soportados (rechazo explícito).
- **Datos no se copian en migración de PVC:** el sistema se niega a operar si el PVC no parece vacío.
- **Una réplica del operador:** leader election está deshabilitada por defecto. Para alta disponibilidad habilitarla en `cmd/main.go`.
- **Requiere BTF en el kernel:** kernels antiguos sin BTF no son compatibles con la versión actual basada en `tp_btf`.

Líneas de trabajo futuro

1. Integración con `VolumeSnapshot` + `Job` de copia para migraciones de

- almacenamiento no destructivas.
2. Soporte de `StatefulSet` y `DaemonSet` en remediación.
 3. Plano horizontal: combinar escalado vertical in-place con HPA cuando vertical agota margen.
 4. Webhook de admisión que ajuste *requests* iniciales basándose en histórico.
 5. Métrica adicional: latencia de red de bajo nivel (TCP retransmits, queueing en sockets).
 6. Empaquetado como Helm chart con CRDs versionados.

13. Apéndices

Apéndice A — Comandos útiles

Tarea	Comando
Ver programas BPF cargados	<code>sudo bpftool prog show</code>
Volcar un mapa BPF	<code>sudo bpftool map dump pinned /sys/fs/bpf/block_latency/stats_by_cgroup</code>
Ver <i>printk</i> de eBPF	<code>sudo cat /sys/kernel/debug/tracing/trace_pipe</code>
Ver métricas del collector	<code>curl http://localhost:9100/metrics</code>
Estado del CR	<code>kubectl describe ioremediationpolicy <nombre></code>
Logs del operador	<code>kubectl logs -n <ns> deploy/operator-controller-manager -f</code>
Activar dashboard TUI del collector	<code>COLLECTOR_DEBUG_TUI=1 ./collector</code>

Apéndice B — Troubleshooting

Síntoma	Causa probable	Solución
El collector no encuentra <code>stats_by_cgroup</code>	Programas eBPF no cargados o no pineados	Re-ejecutar <code>start_system</code> y comprobar <code>/sys/fs/bpf/</code>
<code>name_to_handle_at</code> devuelve más de 8 bytes	cgroupfs v1 o filesystem inesperado	Asegurar cgroups v2 unificados
El operador devuelve <code>forbidden</code>	RBAC sin regenerar	<code>make manifests && make deploy</code>
ScaleUp falla con "pod updates may not change..."	K8s < 1.27 o feature gate desactivado	Activar <code>InPlacePodVerticalScaling</code>
La query Prometheus devuelve siempre vacío	Threshold mal interpretado (ms vs ns)	Revisar logs del evaluador, la query se imprime al ejecutarse
Series Prometheus con huecos	Pods recreados; los counters se resetean	El delta-tracking en el collector lo absorbe; revisar pod churn

Apéndice C — Glosario

BTF

BPF Type Format: información de tipos del kernel embebida, base de CO-RE.

cgroup

Control group, mecanismo del kernel para limitar y contabilizar recursos.

CO-RE

Compile Once - Run Everywhere: técnica para portabilidad de eBPF.

CRD

Custom Resource Definition: extensión declarativa de la API de Kubernetes.

HPA / VPA

Horizontal/Vertical Pod Autoscaler.

PVC

PersistentVolumeClaim: petición de almacenamiento persistente en Kubernetes.

Run Queue

Estructura por CPU donde el scheduler encola tareas *runnable*.

Tracepoint

Punto de instrumentación estable expuesto por el kernel.

tp_btf

Variante de tracepoint con argumentos tipados via BTF; permite usar BPF_PROG.